GDI-DE | Geodateninfrastruktur Deutschland

Übung: Geodatenbasierter Zuständigkeitsfinder

Praxisübung · Python · Jupyter Notebook · WFS · GeoPandas · Leafmap · WMS · CSW

Szenario

Welche Behörde oder öffentliche Stelle ist für eine bestimmte Adresse zuständig? Entwickelt einen geodatenbasierten Zuständigkeitsfinder. Als Einstieg verwendet ihr sieben Zuständigkeitsbereiche der **Bundesagentur für Arbeit und der Gerichte** aus dem offenen WFS *Behördenzuständigkeitsbereiche Open* des Bundesamts für Kartographie und Geodäsie (BKG). Die Lösung soll grundsätzlich auf weitere Behördenarten und später auch auf andere WFS-Anbieter übertragbar sein.

Ihr untersucht den WFS, ladet Zuständigkeitsflächen, ermittelt die Position einer Adresse und prüft, in welchem Zuständigkeitsgebiet sie liegt. Anschließend erstellt ihr eine interaktive Karte und exportiert eine **eigenständige HTML-Webanwendung** im abgestimmten GDI-DE-Design (türkis-transparente Zuständigkeitsflächen, schwarze Überschriften und türkise Bedienelemente) mit Adresssuche und dem BKG-WMS **TopPlusOpen Grau** als Hintergrundkarte.

Lernziele

- Einen WFS über **GetCapabilities** untersuchen, alle angebotenen Feature-Typen entdecken und einen auswählen.
- Geodaten als **GeoJSON** abrufen, in GeoPandas verarbeiten und das Koordinatensystem prüfen.
- Adressen mit **Nominatim** geokodieren und den Treffer kontrollieren.
- Mit **Shapely/GeoPandas** die räumlich zuständige Behörde ermitteln.
- Zuständigkeitsgebiete und Suchergebnis in **Leafmap/Folium** visualisieren.
- Eine eigenständige **HTML-Anwendung** mit WMS-Hintergrundkarte, Adresssuche, dynamischer Layerauswahl und GeoJSON-Export aus dem Notebook erzeugen.
- Als Zusatzaufgabe **ISO-Metadaten über CSW** und die zugehörigen Nutzungsbedingungen auswerten.

Datenquellen und Hinweise

- BKG WFS Behördenzuständigkeitsbereiche Open – Dienstinformation
- WFS GetCapabilities
- BKG WMS TopPlusOpen – Dienstinformation
- BKG-Datenquellenverzeichnis
- Geodatenkatalog.de
- Nominatim-Such-API · Nutzungsrichtlinie

Hinweis: Verwendet für die Adresssuche möglichst eine vollständige Adresse mit Straße und Hausnummer. Prüft den gefundenen Geokodierungstreffer. Die öffentliche Nominatim-API ist nicht für umfangreiche oder automatisierte Massenabfragen vorgesehen.

Arbeitsauftrag

Bearbeitet die Abschnitte **1–8** der Reihe nach. Führt die Python-Zellen aus und ergänzt die markierten Aufgaben. Prüft zunächst beispielhaft **Mainz**, verwendet anschließend eine weitere Adresse und vergleicht die ermittelte Zuständigkeit. Erstellt zum Abschluss aus dem Notebook die Datei `Zustaendigkeitsfinder_hack4GDI_DE_2026.html` und testet sie im Browser.

Die bereitgestellte HTML-Vorlage bildet das gewünschte Design bereits ab. Eure Aufgabe ist es, die **Geodatenverarbeitung nachzuvollziehen**, die Konfiguration zu prüfen und die Anwendung korrekt zu erzeugen – nicht, die gesamte JavaScript-Oberfläche neu zu programmieren.

Zusatzaufgabe · Metadaten und Quellenvermerk

Recherchiert den **ISO-19139-Metadatensatz** des WFS über die CSW-Schnittstelle des Geodatenkatalog.de. Gebt Titel, Metadatensatz-ID, Nutzungsbedingungen, Zugriffsbeschränkungen und Lizenzverweise aus; kennzeichnet fehlende Angaben ausdrücklich. Vergleicht den Quellenvermerk des Metadatensatzes mit den Angaben in der HTML-Anwendung.

Die Kartenattribution soll **Leaflet | Kartengrundlage: © Bundesamt für Kartographie und Geodäsie (2026) · dl-de/by-2-0** enthalten. Die weiterführenden Datenquellen stehen im Footer der Webanwendung.

Verfügbare Bereiche: Arbeitsagenturen, Geschäftsstellen, Jobcenter, Regionaldirektionen, Amtsgerichte, Landgerichte und Oberlandesgerichte. Die HTML-Anwendung liest die aktuelle Layerliste über GetCapabilities ein; bei Ausfall werden diese sieben Layer als Rückfalloption angeboten.

1 · Bibliotheken installieren

In [ ]:
```
!pip install -q requests geopandas shapely leafmap folium owslib pyogrio beautifulsoup4
```

2 · Konfiguration

In [ ]:
```
import requests
import geopandas as gpd
import leafmap.foliumap as leafmap
import folium
from shapely.geometry import Point
from IPython.display import display

WFS_URL = "https://sgx.geodatenzentrum.de/wfs_bzb_open"
LAYER = "bzb-open:ba_arbeitsagenturen"  # Zuständigkeitsbereiche der Arbeitsagenturen
ADRESSE = "Mainz, Rheinland-Pfalz, Deutschland"  # besser eine vollständige Straßenadresse
USER_AGENT = "hack4GDI-DE-Zustaendigkeitsfinder/1.0 (kontakt: jupyter@hack4gdi.de)"
```

3 · WFS-GetCapabilities auswerten

In [ ]:
```
# Aufgabe 3: Stellt mit OWSLib eine Verbindung zum WFS her.
# Gebt Diensttitel, Anbieter und die verfügbaren Feature-Typen aus.
# Prüft, ob LAYER vorhanden ist.
```

4 · Zuständigkeitsflächen als GeoJSON abrufen

In [ ]:
```
# Aufgabe 4: Ruft den Feature-Typ LAYER über WFS GetFeature als GeoJSON ab.
# Erstellt daraus ein GeoDataFrame gdf und prüft Anzahl, Attribute, CRS und Bounds.
# Achtet auf die korrekte Interpretation von EPSG:4326.
```

5 · Adresse geokodieren

In [ ]:
```
# Aufgabe 5: Fragt ADRESSE über Nominatim ab.
# Prüft den Treffer und erstellt aus lon/lat einen Shapely-Punkt `punkt`.
```

6 · Zuständigkeitsbereich bestimmen

In [ ]:
```
# Aufgabe 6: Bestimmt mit `gdf.geometry.covers(punkt)` die passenden Flächen.
# Speichert das Ergebnis als `treffer` und gebt Name und Anzahl aus.
```

7 · Interaktive Karte im abgestimmten Farbschema

In [ ]:
```
# Aufgabe 7: Erstellt eine Leafmap/Folium-Karte mit TopPlusOpen Grau (WMS).
# Zeichnet alle Zuständigkeitsbereiche in Türkis (#008b95) mit transparenter Füllung, den Treffer hervorg
# Hinterlegt den BKG-Quellenvermerk direkt in der Kartenattribution.
# Speichert die Karte als `zustaendigkeitsfinder_karte.html`.
```

8 · Finale HTML-Anwendung aus dem Notebook erzeugen

Aufgabe: Erstellt die vollständige Webanwendung **„Geodatenbasierter Zuständigkeitsfinder“** direkt aus dem Notebook. Die fertige HTML-Vorlage ist bereits eingebettet; eine zusätzliche Datei wird nicht benötigt.

1. Prüft die Einstellungen `WFS_BROWSER`, `STANDARD_LAYER`, `STANDARD_HINTERGRUND` und `BEISPIEL_ADRESSE`.
2. Verwendet **TopPlusOpen Grau** als Standardhintergrund. Die Karte wird über den **BKG-WMS** geladen.
3. Führt die Zelle aus, öffnet die erzeugte HTML-Datei im Browser und testet die Suche mit Mainz und einer zweiten Adresse.
4. Prüft die dynamische Layerauswahl (WFS-GetCapabilities), die Kartenattribution, den Footer und den GeoJSON-Export. Vergleicht außerdem die Darstellung mit der freigegebenen HTML-Vorlage: rote GDI-DE-Navigation, schwarze Überschriften, türkis-transparente Zuständigkeitsflächen, türkiser Suchbutton sowie mittelgraue Hintergrundkartenauswahl.

Hinweis: Die HTML-/JavaScript-Vorlage ist für diese Übung vorbereitet. Die Konfiguration könnt ihr anpassen; für weitere WFS-Anbieter wäre später eine Erweiterung der Layer-Konfiguration nötig.

Gestaltung: Die HTML-Datei wird aus der final abgestimmten Vorlage erzeugt. Änderungen an den Farben erfolgen bei Bedarf im eingebetteten HTML/CSS, nicht an der Geodatenverarbeitung.

In [ ]:
```
# Konfiguration für die aus dem Notebook erzeugte Webanwendung.
# Aufgabe: Prüft oder ändert die folgenden Einstellungen.
WFS_BROWSER = "https://sgx.geodatenzentrum.de/wfs_bzb_open"
STANDARD_LAYER = "bzb-open:ba_arbeitsagenturen"
STANDARD_HINTERGRUND = "grau"  # "grau" oder "farbe"
BEISPIEL_ADRESSE = "Mainz, Rheinland-Pfalz, Deutschland"
HTML_DATEI = "Zustaendigkeitsfinder_hack4GDI_DE_2026.html"

# Die finale HTML-Vorlage „Geodatenbasierter Zuständigkeitsfinder“ ist platzsparend im Notebook enthalten
# So kann das Notebook die vollständige HTML-Datei ohne zusätzliche Vorlagendatei erstellen.
# Für die Übung müsst ihr den HTML-/JavaScript-Quelltext nicht vollständig neu schreiben.
import base64
import zlib
from pathlib import Path
from html import escape
from IPython.display import IFrame, display, FileLink

_VORLAGE_KOMPRIMIERT = 'eNqUvFmS49a2JfivUXjqWZrlKyiEvmFIupUgQDQkOgIgup809H1D9IBMZjWHGkD95Biyfuqr3kxqJAW6R

def erstelle_html_datei():
    if STANDARD_HINTERGRUND not in ("grau", "farbe"):
        raise ValueError("STANDARD_HINTERGRUND muss 'grau' oder 'farbe' sein")
    if STANDARD_LAYER not in (
        "bzb-open:ba_arbeitsagenturen",
        "bzb-open:ba_geschaeftsstellen",
        "bzb-open:ba_regionaldirektionen",
        "bzb-open:ba_jobcenter",
        "bzb-open:gerichte_amtsgerichte",
        "bzb-open:gerichte_landgerichte",
        "bzb-open:gerichte_oberlandesgerichte",
    ):
        raise ValueError("STANDARD_LAYER ist nicht in der HTML-Vorlage enthalten")

    vorlage = zlib.decompress(base64.b64decode(_VORLAGE_KOMPRIMIERT)).decode("utf-8")
    vorlage = vorlage.replace(
        'const WFS="https://sgx.geodatenzentrum.de/wfs_bzb_open";',
        'const WFS=' + __import__('json').dumps(WFS_BROWSER, ensure_ascii=True) + ';',
    )
    # Das ausgewählte <option>-Element wird zum Standard.
    import re
    vorlage = re.sub(r'(<option value="[^"]+") selected', r'\1', vorlage)
    vorlage = vorlage.replace('<option value="' + STANDARD_LAYER + '">',
                              '<option value="' + STANDARD_LAYER + '" selected>')
    vorlage = vorlage.replace('value="Mainz, Rheinland-Pfalz, Deutschland"',
                              'value="' + escape(BEISPIEL_ADRESSE, quote=True) + '"')
    if STANDARD_HINTERGRUND == "farbe":
        vorlage = vorlage.replace('grau.addTo(map);', 'farbe.addTo(map);')
    datei = Path(HTML_DATEI)
    datei.write_text(vorlage, encoding="utf-8")
```

```
    print(f"HTML-Datei erstellt: {datei.resolve()} ({datei.stat().st_size:,} Bytes)")
    display(FileLink(str(datei)))
    return datei

html_pfad = erstelle_html_datei()
```

HTML-Anwendung testen

Öffnet die erzeugte HTML-Datei über den Dateibrowser von JupyterLab oder über den ausgegebenen Downloadlink im Browser. Die folgende Vorschau kann in JupyterHub wegen Browser-Sicherheitsrichtlinien für externe Dienste eingeschränkt sein; die heruntergeladene HTML-Datei ist der maßgebliche Test.

In [ ]:
```
# Vorschau im Notebook (optional):
# display(IFrame(src=html_pfad.name, width="100%", height=700))
print("Bitte die HTML-Datei im Browser öffnen und die Adresssuche testen.")
```

Zusatzaufgabe · Metadatenabfrage über CSW

Recherchiert den ISO-19139-Metadatensatz des WFS über den Geodatenkatalog.de und gebt die tatsächlich hinterlegten Nutzungsbedingungen, Zugriffsbeschränkungen und Lizenzverweise aus. Vergleicht den Quellenvermerk aus dem Metadatensatz mit der für die Karte vorgegebenen Attribution.

In [ ]:
```
# Zusatzaufgabe: Recherchiert über den CSW des Geodatenkatalog.de den
# ISO-19139-Metadatensatz des WFS.
# Gebt Titel, ID, Nutzungsbedingungen, Zugriffsbeschränkungen und
# Lizenzverweise aus. Vergleicht die Quellenangaben mit der HTML-Anwendung.
```